DATABASE SYSTEMS

SEMESTER PROJECT REPORT

VEI NEXUS

Learning Management System

A Full-Stack LMS for Academic Institution

Submitted By

Student ID	Name	Contribution
24K-0961	**Mahad Khan**	Frontend & Integration
24K-0884	**Shahbaz Hassan**	Backend & API
24K-1039	**Samad Siddique**	Database & Schema

Course: Database Systems (DBS)

Spring Semester 2026

1. Project Title

VEI NEXUS — Learning Management System (LMS)

VEI NEXUS is an institutional Learning Management System designed and developed as a full-stack web application that bridges students and teachers through a centralized digital platform.

2. Group Members

Student ID	Full Name	Role / Contribution
24K-0961	Mahad Khan	Frontend Development & System Integration
24K-0884	Shahbaz Hassan	Backend Development & REST API
24K-1039	Samad Siddique	Database Design & Schema Normalization

3. Project Description

3.1 Overview

VEI NEXUS is a web-based Learning Management System developed specifically for academic institutions. The system provides a unified digital environment where students and teachers can interact through structured academic workflows. The platform automates routine academic operations — including student registration, attendance tracking, and grade management — reducing administrative overhead and improving transparency for all stakeholders.

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User interface and client-side logic
Backend	Node.js (Express.js)	RESTful API and server-side processing
Database	Supabase (PostgreSQL)	Relational database and real-time data layer
Authentication	Supabase Auth + JWT	Secure session and role-based access control
Integration	Full-Stack (Frontend-Backend-DB)	All layers fully integrated end-to-end

3.3 System Modules

Student Module

- Login with institute-allocated credentials (ID and password)
- View personal academic dashboard
- Monitor attendance records across all enrolled courses
- View grades and marks submitted by teachers
- Self-registration using a pre-allocated student ID

Teacher Module

- Secure teacher login via credential-based authentication
- Mark and manage student attendance for each course session
- Enter, update, and manage student grades and marks
- View class roster and enrolled student list
- Monitor overall class performance at a glance

Admin / System Module

- Institute-level management of users, courses, and enrollments
- Allocation of student and teacher IDs and credentials
- Course creation and teacher-course assignment

3.4 Key Features

- Role-based access control: separate portals for students and teachers
- Real-time data synchronization through Superbase's PostgreSQL backend
- Secure authentication with hashed passwords and JWT session tokens
- Fully integrated frontend-backend-database architecture
- Responsive web interface accessible across devices
- Attendance tracking with Present / Absent / Leave status per session
- Grade management with marks, totals, and letter grades

4. Project Requirements

4.1 Functional Requirements

FR-01: User Authentication

- The system shall allow students to register using an institute-allocated student ID
- The system shall authenticate users (students and teachers) via email/ID and password
- The system shall enforce role-based access — students cannot access teacher features and vice versa

FR-02: Student Features

- Students shall be able to view their attendance records per course
- Students shall be able to view their grades and marks per course
- Students shall be able to view their enrolled courses

FR-03: Teacher Features

- Teachers shall be able to mark attendance for students on a per-session basis
- Teachers shall be able to enter and update grades for enrolled students
- Teachers shall be able to view the list of students enrolled in their courses

FR-04: Course Management

- The system shall support course creation with a title, code, and assigned teacher
- The system shall support student enrollment into courses
- Each course shall maintain its own attendance and grade records independently

FR-05: Data Integrity

- All foreign key relationships shall be enforced at the database level
- Duplicate enrollment of the same student in the same course shall be prevented
- Attendance records shall be unique per student per course per date

4.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Passwords must be stored as hashed values (bcrypt). JWT tokens used for session management.
NFR-02	Performance	API responses must be returned within 2 seconds under normal load conditions.
NFR-03	Scalability	The database schema must support multiple institutes, courses, and hundreds of students.
NFR-04	Usability	The interface must be intuitive and accessible on both desktop and mobile browsers.
NFR-05	Reliability	Supabase managed PostgreSQL ensures 99.9% uptime with automated backups.
NFR-06	Maintainability	Codebase must follow modular structure with separate frontend, backend, and DB layers.

4.3 Hardware & Software Requirements

Development Environment

- Operating System: Windows 10/11 or macOS
- Node.js v18+ for backend runtime

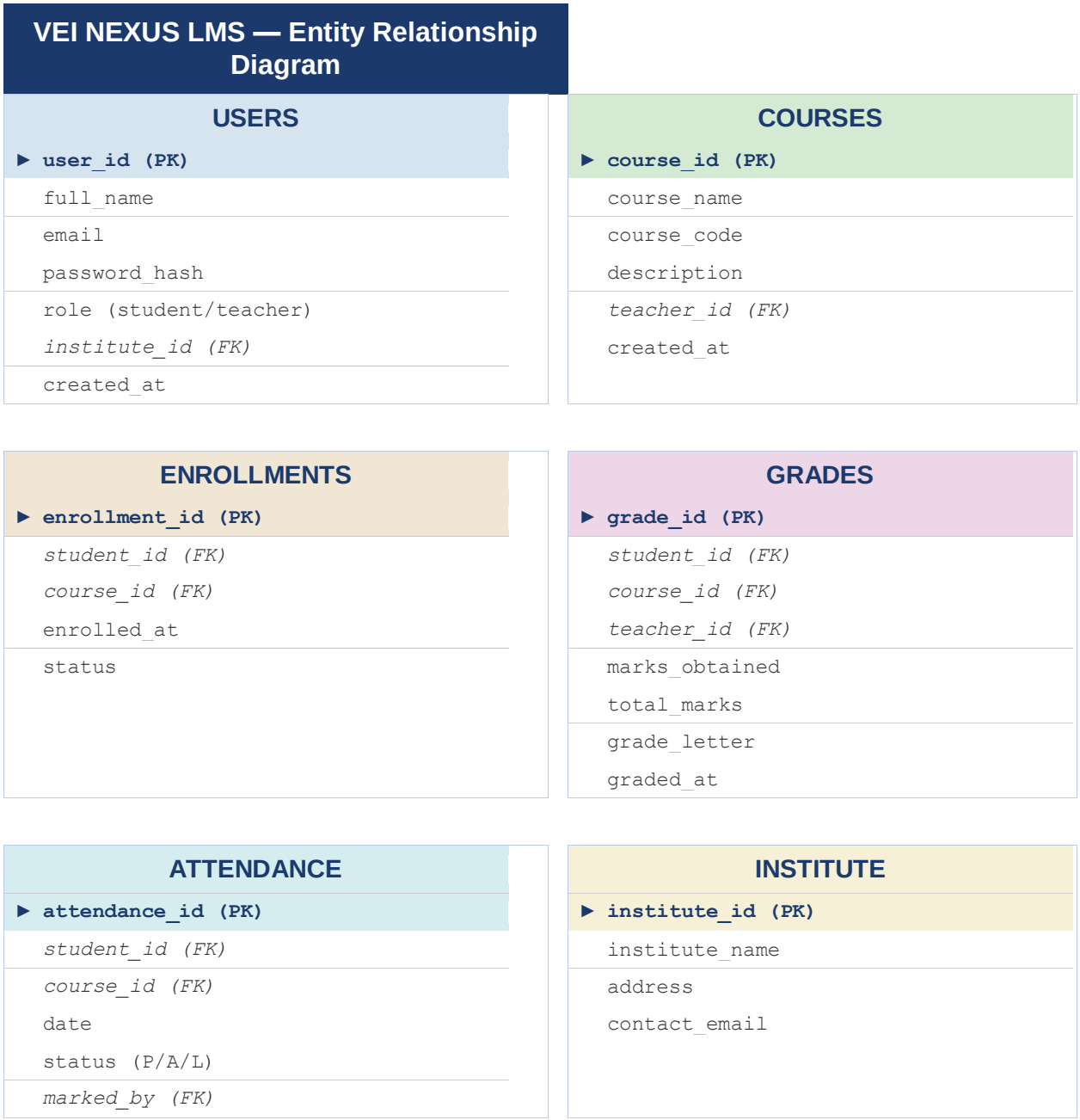
- Web Browser (Chrome / Firefox / etc.) for frontend testing
- VS Code as the primary IDE
- Git for version control

Deployment Environment

- Supabase Cloud for PostgreSQL database hosting
- Node.js server for backend API hosting
- Static file hosting for HTML/CSS/JS frontend

5. Entity-Relationship (ER) Diagram

The following diagram illustrates the six core entities of the VEI NEXUS database, their attributes, and the relationships between them. Primary Keys (PK) are highlighted and Foreign Keys (FK) are italicized.



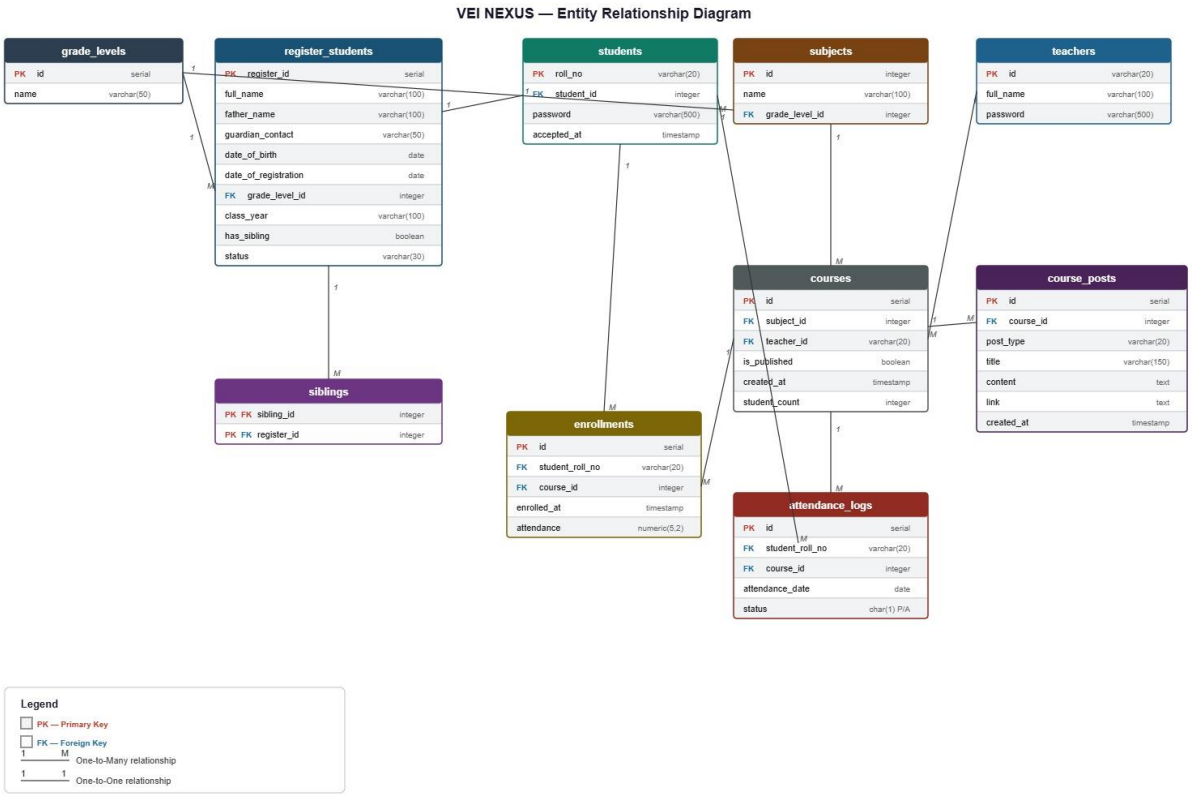
5.1 Entity Descriptions

Entity	Description	Primary Key
USERS	Stores all system users (students and teachers) with role-based differentiation	user_id
COURSES	Represents courses offered by the institute, each assigned to one teacher	course_id
ENROLLMENTS	Junction table linking students to the courses they are enrolled in	enrollment_id
GRADES	Records marks and letter grades assigned by teachers to students per course	grade_id
ATTENDANCE	Tracks daily attendance status (Present/Absent/Leave) per student per course	attendance_id
INSTITUTE	Master table for institute information; users belong to an institute	institute_id

5.2 Relationships

- INSTITUTE (1) ----< USERS (M): One institute has many users (students and teachers)
- USERS/Teacher (1) ----< COURSES (M): One teacher teaches many courses
- USERS/Student (M) >----< COURSES (M) via ENROLLMENTS: Many students enroll in many courses
- USERS/Student (1) ----< GRADES (M): One student has many grade records
- COURSES (1) ----< GRADES (M): One course has many grade entries
- USERS/Student (1) ----< ATTENDANCE (M): One student has many attendance entries
- COURSES (1) ----< ATTENDANCE (M): One course has many attendance entries

5.3 Relationships



6. Normalized Database Schema

The VEI NEXUS database is designed in Third Normal Form (3NF). Each table satisfies the requirements of 1NF, 2NF, and 3NF as described below.

6.1 Normalization Overview

Normal Form	Rule	Status in VEI NEXUS
1NF	All attributes are atomic; no repeating groups	Satisfied — each column holds a single, indivisible value
2NF	No partial dependencies on composite primary keys	Satisfied — all tables use single-column surrogate PKs
3NF	No transitive dependencies between non-key attributes	Satisfied — each non-key attribute depends only on the PK
BCNF	Every determinant is a candidate key	Largely satisfied; minor deviations accepted for practicality

6.2 Table Schemas

Table 1: institute

Column	Data Type	Constraint	Description
institute_id	UUID / SERIAL	**PK NOT NULL**	Unique identifier for each institute
institute_name	VARCHAR (200)	NOT NULL	Full name of the institute
address	TEXT	NULLABLE	Physical address of the institute
contact_email	VARCHAR (150)	UNIQUE NOT NULL	Official contact email

Table 2: users

Column	Data Type	Constraint	Description
user_id	UUID / SERIAL	**PK NOT NULL**	Unique identifier for each user
full_name	VARCHAR(150)	NOT NULL	Full name of the user
email	VARCHAR(150)	UNIQUE NOT NULL	Login email address
password_hash	TEXT	NOT NULL	Bcrypt-hashed password
role	ENUM('student','teacher')	NOT NULL	User role in the system
institute_id	UUID / INT	*FK -> institute NOT NULL*	Reference to the user's institute
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

Table 3: courses

Column	Data Type	Constraint	Description
course_id	UUID / SERIAL	**PK NOT NULL**	Unique identifier for each course
course_name	VARCHAR(200)	NOT NULL	Full name of the course
course_code	VARCHAR(20)	UNIQUE NOT NULL	Short course code (e.g. CS-301)
description	TEXT	NULLABLE	Course description or syllabus summary
teacher_id	UUID / INT	*FK -> users NOT NULL*	Teacher assigned to this course
created_at	TIMESTAMP	DEFAULT NOW()	Course creation timestamp

Table 4: enrollments

Column	Data Type	Constraint	Description
enrollment_id	UUID / SERIAL	**PK NOT NULL**	Unique enrollment record ID
student_id	UUID / INT	*FK -> users NOT NULL*	Reference to the enrolled student
course_id	UUID / INT	*FK -> courses NOT NULL*	Reference to the enrolled course
enrolled_at	TIMESTAMP	DEFAULT NOW()	Date and time of enrollment
status	VARCHAR(20)	DEFAULT 'active'	Enrollment status (active/dropped)
(student_id, course_id)	—	UNIQUE	Prevents duplicate enrollments

Table 5: grades

Column	Data Type	Constraint	Description
grade_id	UUID / SERIAL	**PK NOT NULL**	Unique grade record ID
student_id	UUID / INT	*FK -> users NOT NULL*	Reference to the student being graded
course_id	UUID / INT	*FK -> courses NOT NULL*	Reference to the course
teacher_id	UUID / INT	*FK -> users NOT NULL*	Teacher who entered the grade
marks_obtained	DECIMAL(5,2)	NOT NULL	Marks scored by the student
total_marks	DECIMAL(5,2)	NOT NULL	Total possible marks
grade_letter	CHAR(2)	NULLABLE	Letter grade (A, B+, C, etc.)

Column	Data Type	Constraint	Description
graded_at	TIMESTAMP	DEFAULT NOW()	When the grade was entered

Table 6: attendance

Column	Data Type	Constraint	Description
attendance_id	UUID / SERIAL	`PK NOT NULL`	Unique attendance record ID
student_id	UUID / INT	*FK -> users NOT NULL*	Reference to the student
course_id	UUID / INT	*FK -> courses NOT NULL*	Reference to the course
date	DATE	NOT NULL	Date of the class session
status	CHAR(1)	CHECK IN ('P','A','L')	P=Present, A=Absent, L=Leave
marked_by	UUID / INT	*FK -> users NOT NULL*	Teacher who marked attendance
(student_id, course_id, date)	—	UNIQUE	One record per student per session

6.3 Foreign Key Relationships Summary

Foreign Key	From Table	References	On Delete
institute_id	users	institute(institute_id)	RESTRICT
teacher_id	courses	users(user_id)	RESTRICT
student_id	enrollments	users(user_id)	CASCADE
course_id	enrollments	courses(course_id)	CASCADE
student_id	grades	users(user_id)	CASCADE
course_id	grades	courses(course_id)	CASCADE
teacher_id	grades	users(user_id)	RESTRICT
student_id	attendance	users(user_id)	CASCADE
course_id	attendance	courses(course_id)	CASCADE
marked_by	attendance	users(user_id)	RESTRICT

7. Conclusion

VEI NEXUS successfully demonstrates the design and implementation of a relational database-driven web application for academic management. The system was built using a modern full-stack architecture — HTML/CSS/JavaScript on the frontend, Node.js on the backend, and Supabase

(PostgreSQL) as the cloud-hosted relational database. The database schema is normalized to the Third Normal Form (3NF), ensuring data integrity, eliminating redundancy, and maintaining referential consistency through well-defined foreign key relationships.

The project covers all fundamental DBS concepts including entity-relationship modeling, schema normalization, constraint enforcement, and relational query design. The complete integration of all three tiers (presentation, business logic, and data) demonstrates a production-ready application architecture capable of supporting real institutional workflows.

This project was a collaborative effort by Mahad Khan (24K-0961), Shahbaz Hassan (24K-0884), and Samad Siddique (24K-1039), submitted in partial fulfillment of the Database Systems (DBS) course requirements for the Spring Semester 2024-25.